

Clase 090 — Mapeo, spidering y descubrimiento de contenido

Parte: **4 — Seguridad de aplicaciones web** · Fuente: *The Web Application Hacker's Handbook / Bug Bounty Bootcamp (Vickie Li)* 🕒 Duración estimada: **100 min** · Nivel: **Intermedio**

🎯 Objetivo

Aprender a **mapear exhaustivamente** una aplicación: descubrir directorios, archivos, endpoints ocultos, parámetros y funciones no enlazadas. Un buen mapeo multiplica la superficie de ataque real y suele ser la diferencia entre encontrar un bug crítico o no encontrar nada.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Combinar** spidering pasivo y activo para construir el sitemap.
2. **Aplicar** fuzzing de contenido con diccionarios (dirbusting) de forma eficiente.
3. **Descubrir** parámetros ocultos y endpoints de API no enlazados.
4. **Enumerar** subdominios y virtual hosts relevantes al target.
5. **Priorizar** los hallazgos por probabilidad de contener vulnerabilidades.

📖 Temas

#	Tema	Por qué importa
1	Spidering pasivo vs. activo	Base del descubrimiento
2	Content discovery (dirbusting)	Encuentra lo no enlazado
3	Diccionarios (SecLists)	Calidad de wordlist = calidad de hallazgos
4	Descubrimiento de parámetros	Inputs ocultos = bugs ocultos
5	Enumeración de subdominios	Amplía el scope legítimo
6	Análisis de JavaScript	Los JS revelan rutas de API
7	robots.txt, sitemap.xml, .git	Fuentes de rutas sensibles

📖 Definiciones y características

- **Content discovery:** proceso de encontrar recursos no enlazados por fuerza bruta de rutas. Característica: depende de un buen diccionario.
- **Wordlist:** lista de rutas/palabras candidatas. Característica: SecLists es el estándar de facto.
- **Fuzzing de parámetros:** probar nombres de parámetros para descubrir los ocultos. Característica: cambia la respuesta si el parámetro existe.

- **Subdominio**: host bajo el dominio principal. Característica: puede quedar fuera de las protecciones del principal.
- **Virtual host (vhost)**: sitios distintos servidos por la misma IP según la cabecera Host. Característica: se enumeran fuzzando el Host.
- **Recursión de directorios**: repetir el dirbusting dentro de cada carpeta hallada. Característica: descubre estructuras profundas.

Herramientas y preparación

- **ffuf**, **feroxbuster** o **gobuster** para content discovery.
- **SecLists** (diccionarios).
- **Arjun** para descubrimiento de parámetros; **subfinder**/**amass** para subdominios.
- **LinkFinder**/**gau** para extraer rutas del JavaScript.

```
sudo apt install ffuf gobuster
git clone https://github.com/danielmiessler/SecLists
pipx install arjun
```

Laboratorio guiado

⚠ Solo contra tu propio laboratorio (Juice Shop / DVWA) o programas con permiso explícito.

1. Levanta Juice Shop y hazlo pasar por Burp para poblar el sitemap con navegación manual.
2. Ejecuta content discovery con ffuf:

```
ffuf -u http://localhost:3000/FUZZ -w SecLists/Discovery/Web-Content/common.txt -mc 200,301,3
```

3. Revisa `robots.txt`, `sitemap.xml` y `/ftp` (una ruta famosa de Juice Shop).
4. Extrae rutas embebidas en los archivos `.js` con LinkFinder o grep de patrones `\.rest\.`
5. Descubre parámetros ocultos en un endpoint con Arjun:

```
arjun -u http://localhost:3000/rest/products/search
```

6. Fuzzea la extensión de archivos (`FUZZ.php`, `FUZZ.bak`) buscando backups.
7. Consolida todo en un sitemap enriquecido y marca los endpoints prometedores.

Ejercicios

1. Compara resultados de `common.txt` vs. `directory-list-2.3-medium.txt`.
2. Usa filtros por tamaño (`-fs`) para eliminar respuestas de "not found" personalizadas.
3. Encuentra un endpoint de API en el JS que no aparece navegando.
4. Enumera subdominios de un dominio propio con subfinder.
5. Descubre al menos 3 parámetros ocultos en un endpoint de Juice Shop.
6. Explica por qué un `403` puede ser más interesante que un `404`.

Reto verificable

Construye un **inventario de endpoints** de Juice Shop que incluya al menos 5 rutas no descubribles solo navegando (obtenidas por dirbusting, análisis de JS o parámetros ocultos). **Criterio de aceptación:** cada ruta se acompaña de cómo se descubrió (herramienta + evidencia) y una hipótesis de por qué podría ser vulnerable.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Todo devuelve 200	La app tiene "soft 404"; filtra por tamaño con <code>-fs</code>
Escaneo eterno	Diccionario demasiado grande; empieza por <code>common.txt</code>
Rate limiting / bloqueo	Baja hilos (<code>-t</code>) y añade delays
No aparecen rutas de API	Analiza el JavaScript, no solo el HTML
Fuzzear fuera de scope	Limita el target a hosts autorizados

Preguntas frecuentes

 **¿Qué diccionario uso?** Empieza con `common.txt` para rapidez; escala a `directory-list-2.3-medium` si necesitas cobertura. Ajusta al stack (rutas PHP, ASPX, etc.).

 **¿Por qué analizar el JavaScript?** En SPAs, la mayoría de endpoints de API están referenciados en los bundles JS, no en el HTML navegable.

 **¿El content discovery es intrusivo?** Genera muchas peticiones y puede disparar alertas o rate limits. Hazlo solo con autorización y controlando la velocidad.

Referencias

- Li, *Bug Bounty Bootcamp*, cap. de reconocimiento.
- SecLists: <https://github.com/danielmiessler/SecLists>
- ffuf: <https://github.com/ffuf/ffuf>
- OWASP WSTG — Content Discovery.

Siguiente clase

Clase 091 - Inyeccion SQL: fundamentos